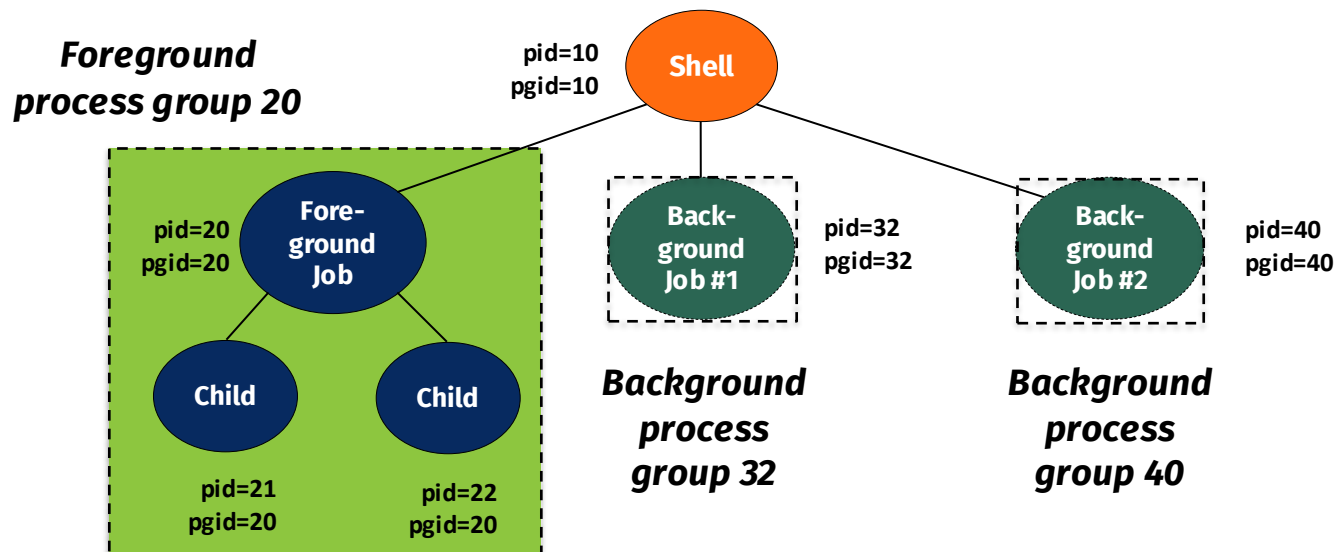


Daemon

Process Execution Type

- **There are two types of processes:**
 - Foreground process
 - Shell must wait for process termination
 - Background process
 - Shell does not wait for process termination
 - One way to generate a background process is appending the '&' symbol to the end of the command line



What is a Daemon?

- **Daemon is a special type of background process**
 - No controlling terminal
 - This is because a process with a controlling terminal can be terminated unintentionally (e.g., logout or press Ctrl + c)
 - Normally start at system boot and keep running forever
 - Some daemons can be started from the user terminal
 - Traditionally, daemon names end with letter 'd'
 - e.g., sshd, crond
 - In Windows, daemons are called service

Daemon Examples

Daemon Name	Function
syslogd	Logging system facility
sshd	Incoming SSH connection service
ftpd	Incoming FTP connection service
crond	Running jobs on a pre-determined schedule
atd	Scheduling jobs with <i>at</i> command
inetd or xinetd	Managing Internet-based services
httpd	Handling HTTP requests

Daemon Coding Rules (1)

- **There are old-style (SysV) daemons and new-style daemons.**
- **Creating old-style SysV daemons requires following certain rules (ref. daemon(7) man page)**
 1. Close all open file descriptors except for standard descriptors.
 2. Reset signal handlers and signal mask to default.
 3. Sanitize environment variables when needed.
 4. Create child process in a new session.
 5. Create daemon process within the child and terminate child.
 6. Within daemon,
 - a) Change standard file descriptors to /dev/null
 - b) Set file mode mask.
 - c) Change the current working directory to the root directory.
 - d) Write the daemon PID to a PID file in a race-free manner.
 - e) Drop privileges when needed
 - f) Notify parent process initialization is done.
 - g) Set exit handler to remove PID file.
 7. Terminate parent process.

Daemon Coding Rules (2)

1. Close all open file descriptors except for standard descriptors 0, 1, 2.

- If /proc/self/fd can be accessed, read this directory's entries, and close the entries that are not 0, 1, 2.

```
void close_fds() {
    DIR *dir = opendir("/proc/self/fd");
    if (dir == NULL) { perror("opendir"); return; }
    int dir_fd = dirfd(dir);
    if (dir_fd == -1) { perror("dirfd"); closedir(dir); return; }
    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {
        if (entry->d_name[0] == '.') continue;
        char *endptr;
        errno = 0;
        uint64_t fd = strtoull(entry->d_name, &endptr, 10);
        if (errno == ERANGE || fd > INT_MAX ||
            *endptr != '\0' || fd < 3 || fd == dir_fd) continue;
        close((int)fd);
    }
    closedir(dir);
}
```

Daemon Coding Rules (3)

2. Reset signal handlers and signal mask to default.

- SIGKILL and SIGSTOP are skipped as behaviour cannot be change
- `int sigprocmask(int how, const sigset_t* set, sigset_t* oldset)`
 - Passing `SIG_SETMASK` to `how` with an empty `set` unblocks all the signals.
 - Returns 0 on success and -1 when an **error occurs**

```
void reset_signal_handlers_and_mask() {  
    struct sigaction sa;  
    memset(&sa, 0, sizeof(sa));  
    sa.sa_handler = SIG_DFL;  
    sigemptyset(&sa.sa_mask);  
    sa.sa_flags = 0;  
    for (int sig = 1; sig < NSIG; sig++) {  
        if (sig == SIGKILL || sig == SIGSTOP) continue;  
        sigaction(sig, &sa, NULL);  
    }  
    sigprocmask(SIG_SETMASK, &sa.sa_mask, NULL);  
}
```

Daemon Coding Rules (4)

3. Sanitize environment variables when needed.

- Restrict the values of the environment variables (PATH, etc.) to desired environment variables.

4a. Generate a child process.

- Call **fork()** and **exit()** system call from parent process
 - Parent process is just a role to create the child process
 - This is needed to create a new session for a daemon and to be adopted by init.

Daemon Coding Rules (5)

4b. Assign child process to a new session.

- `pid_t setsid(void)`
 - Makes the process the leader of the process group and session
 - Returns the session ID of the calling process when it **runs successfully**
 - Returns -1 when an **error occurs**

```
if(fork() == 0) {  
    printf("old session id : %d\n", getsid(getpid()));  
    if(setsid() == -1)  
        printf("setsid failed");  
    else  
        printf("new session id : %d\n", getsid(getpid()));  
}  
wait(NULL);
```

Daemon Coding Rules (6)

5. Create daemon process within child and terminate child.

- The child process creates the session and is therefore the session leader.
- Session leaders can take control of controlling terminal, and once killed, no processes in the session can take control of the terminal.
- By terminating the child, the daemon is adopted by the init process and cannot obtain the controlling terminal.

6a. Change standard file descriptors 0, 1, 2 to /dev/null

- Daemon does not require STDIN, STDOUT, STDERR.
- Many library functions assume that the first three descriptors are open.

Daemon Coding Rules (7)

6b. Resetting file mode mask (*umask*) to 0

- Daemon may want to restrict file permission when creating files.
- If umask value is 1, clear value; otherwise, keep value
 - e.g. value 5 (101), umask 3 (011) → 4 (100)

file permission (passed):	756
umask (permission to restrict):	& 037
resulting file permission:	740

owner:	group:	others:
rwX	r-x	rw-
111	101	110
7	5	6

r	read permission
w	write permission
x	execute permission
-	no permission

6c. Change the current working directory to the root directory.

- Prevents the daemon from blocking the mounted filesystem containing the directory to be unmounted.

Daemon Coding Rules (8)

6d. Write the PID of the daemon process to a PID file in a race-free manner.

- The easiest way is to use LibBSD's `pidfile` (requires passing `-lbsd` and including `bsd/libutil.h`).

```
// parent
struct pidfh* pidfile =
    pidfile_open("/var/run/daemon_name.pid", 0600, NULL);
// ...
if (pid == 0) { // child
    // ...
    if (daemon_pid == 0) { // grandchild (daemon)
        // ...
        pidfile_write(pidfile)
        // ...
        pidfile_close(pidfile)
        // ...
        do_daemon_work();
    } else return EXIT_SUCCESS;
}
```

Daemon Coding Rules (9)

6e. Drop privileges when needed.

- Some operations require root privileges (e.g., writing pid file to /var/run), but staying on root is not safe.
- The daemon should have minimum level of permissions by assigning the process to a user/group with fewer permissions.
 - For daemons, this is usually specific to the daemon itself, but for this example, SUDO_UID and SUDO_GID are used.
- `uid_t geteuid()`: returns effective user id (euid)
- `int setuid(uid_t uid)`: sets user id
- `gid_t setgid(gid_t gid)`: sets group id

```
void drop_privileges(const char* uid_s, const char* gid_s) {  
    if (geteuid() == 0) {  
        CHECK_ERROR_SYSLOG(setgid((gid_t)strtoul(gid_s, NULL, 10)));  
        CHECK_ERROR_SYSLOG(setuid((uid_t)strtoul(uid_s, NULL, 10)));  
    }  
}
```

Daemon Coding Rules (10)

6f. Notify parent process initialization is done.

- Use a pipe to notify the parent that the initialization has finished.

```
// parent
int pipefd[2], pid; pipe(pipefd);
// ...
if (pid == 0) {
    // ...
    if (daemon_pid == 0) {
        // ...
        write(pipefd[1], "1", 1);
        close(pipefd[1]);
    } else return EXIT_SUCCESS;
} else {
    close(pipefd[1]);
    read(pipefd[0], &done, 1);
    close(pipefd[0]);
    return EXIT_SUCCESS;
}
```

Daemon Coding Rules (11)

6g. Set exit handler to remove PID file with atexit.

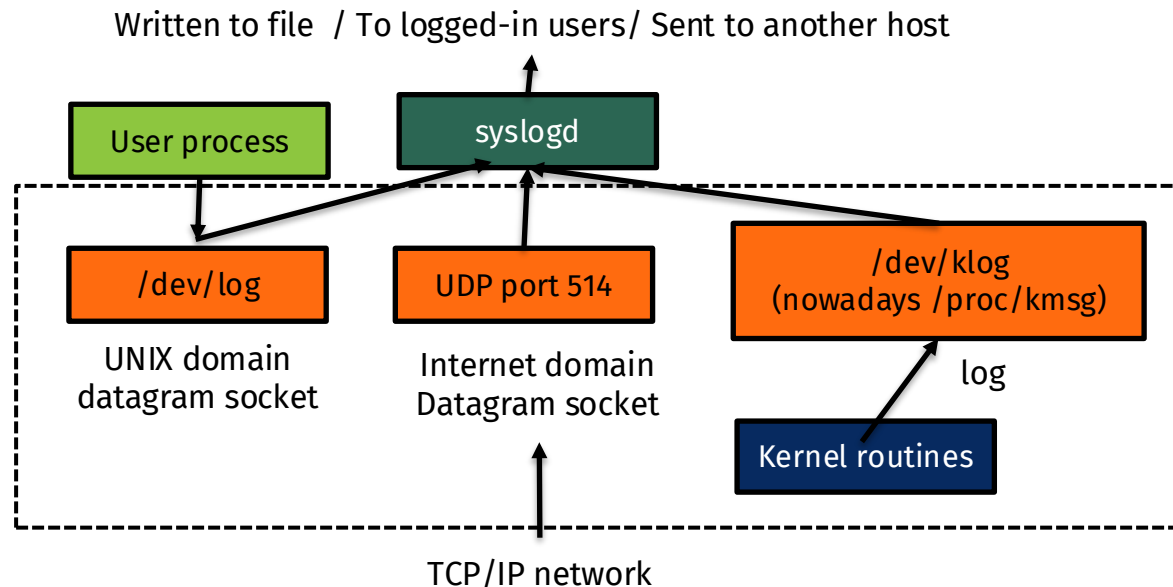
- `int pidfile_remove(struct pidfh *pfh)`
 - Remove PID file.
- `int atexit(void (*function)(void))`
 - Sets handler to run when process terminates.

Logging for Daemon Errors

- **How to handle error messages in daemon?**
 - Daemons don't have a controlling terminal
 - It can't simply write to standard error
 - A central daemon error-logging facility is required!
- **Solution**
 - *syslog* daemon

syslog Daemon

- **There are three ways to communicate with *syslogd* :**
 1. Call the ***syslog()*** function
 2. Send log messages to UDP port 514
 - Only user processes that are connected to the host by a TCP/IP network
 3. Kernel routines



Open & Close System Logger

```
#include <syslog.h>

void openlog(const char* ident, int option, int facility);
void closelog(void);
```

- ***openlog()* opens a connection to the system logger**
 - *ident* is the name of the program
 - *option* is a bitmask specifying various options
 - *facility* is used to specify what type of program logging the message
 - Refer to the configuration file (/etc/rsyslog.d)
 - Logs with different facilities are written to different files
- ***closelog()* closes the file descriptor being used to write to the system logger**

Options for *openlog*

Option	Function
LOG_CONS	Write directly to the system console if there is an error while sending to the system logger
LOG_NDELAY	Open the connection immediately
LOG_NOWAIT	Don't wait for child <i>processes</i> that may have been created while logging the message
LOG_ODELAY	Opening of the connection is delay until <i>syslog()</i> is called
LOG_PERROR	Additionally log the message to stderr
LOG_PID	Include the caller's PID with each message

Syslog Function

- **syslog() generates a log message**
 - *priority* is a combination of the facility and a level
 - *format* argument and any remaining are passed to the **vsprintf()** for formatting
- **setlogmask() sets the log priority mask for the process**
 - Return the previous mask value

```
#include <syslog.h>

void syslog(int priority, const char *format, ...);
int setlogmask(int maskpri);

/* Returns : previous log priority mask value */
```

Syslog Facility & Priority

Facility	Program type
LOG_AUTH	Authorization
LOG_CRON	Cron daemon
LOG_KERN	Kernel
LOG_LPR	Line printer
LOG_MAIL	Mail system
LOG_NEWS	Network news system
LOG_USER	User process
LOG_FTP	File transfer protocol

Priority	Value
LOG_EMERG	0
LOG_ALERT	1
LOG_CRIT	2
LOG_ERR	3
LOG_WARNING	4
LOG_NOTICE	5
LOG_INFO	6
LOG_DEBUG	7

Error Logging Example

```
#include <syslog.h>

void main(void) {
    setlogmask(LOG_MASK(LOG_EMERG));
    openlog("lpd", LOG_PID, LOG_LPR);
    syslog(LOG_EMERG, "Error");
    syslog(LOG_INFO, "Logging");
    closelog();
}
```



```
cat /var/log/syslog | tail -4
Apr  4 18:44:06 spl lpd[99401]: Error
Apr  4 18:44:08 spl lpd[99427]: Error
Apr  4 18:44:08 spl lpd[99440]: Error
Apr  4 18:44:08 spl lpd[99446]: Error
```

Daemon Coding Example

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <dirent.h>
#include <unistd.h>
#include <fcntl.h>
#include <limits.h>
#include <stdint.h>
#include <errno.h>
#include <signal.h>
#include <string.h>
#include <bsd/libutil.h>
#include <sys/stat.h>
#include <syslog.h>

#define ABORT(msg_func, mode, expr, ref_val) do { \
    if ((expr) == ref_val) { \
        msg_func(mode, "%s: %s", #expr, strerror(errno)); \
        exit(EXIT_FAILURE); \
    } \
} while (0)

#define CHECK_ERROR_SYSCALL(expr) ABORT(fprintf, stderr, expr, -1)
#define CHECK_ERROR_STDLIB(expr) ABORT(fprintf, stderr, expr, NULL)
#define CHECK_ERROR_SYSCALL_SYSLOG(expr) ABORT(syslog, LOG_ERR, expr, -1)
#define CHECK_ERROR_STDLIB_SYSLOG(expr) ABORT(syslog, LOG_ERR, expr, NULL)

static struct pidfh* pidfile;
```

Daemon Coding Example

```
void close_fds() {
    DIR *dir;
    CHECK_ERROR_STDLIB(dir = opendir("/proc/self/fd"));
    int dir_fd;
    CHECK_ERROR_SYSCALL(dir_fd = dirfd(dir));
    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {
        if (entry->d_name[0] == '.') continue;
        char *endptr;
        errno = 0;
        uint64_t fd = strtoull(entry->d_name, &endptr, 10);
        if (errno == ERANGE || fd > INT_MAX ||
            *endptr != '\0' || fd < 3 || fd == dir_fd) continue;
        close((int)fd);
    }
    closedir(dir);
}
```

```
void reset_signal_handlers_and_mask() {
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = SIG_DFL;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    for (int sig = 1; sig < NSIG; sig++) {
        if (sig == SIGKILL || sig == SIGSTOP) continue;
        sigaction(sig, &sa, NULL);
    }
    sigprocmask(SIG_SETMASK, &sa.sa_mask, NULL);
}
```


Daemon Coding Example

```
void sanitize_environment_variables() {
    // ...
}

void redirect_to_null_device(){
    int null_fd;
    CHECK_ERROR_SYSCALL(null_fd = open("/dev/null", O_RDWR));
    CHECK_ERROR_SYSCALL(dup2(null_fd, STDOUT_FILENO));
    CHECK_ERROR_SYSCALL(dup2(null_fd, STDIN_FILENO));
    CHECK_ERROR_SYSCALL(dup2(null_fd, STDERR_FILENO));
    CHECK_ERROR_SYSCALL(close(null_fd));
}

void drop_privileges(const char* uid_s, const char* gid_s) {
    // Drop privileges if running as root
    if (geteuid() == 0) {
        uid_t uid = (uid_t)strtoul(uid_s, NULL, 10);
        gid_t gid = (gid_t)strtoul(gid_s, NULL, 10);

        CHECK_ERROR_SYSCALL_SYSLOG(setgid(gid));
        CHECK_ERROR_SYSCALL_SYSLOG(setuid(uid));
    }
}

void notify_parent(int pipefd) {
    CHECK_ERROR_SYSCALL_SYSLOG(write(pipefd[1], "1", 1));
    CHECK_ERROR_SYSCALL_SYSLOG(close(pipefd[1]));
}
```

Daemon Coding Example

```
void do_daemon_work() { while(1); }
void handle_exit() { pidfile_remove(pidfile); }

int main() {
    int fds[5];
    for (int i = 0; i < 5; i++) {
        char filename[NAME_MAX];
        snprintf(filename, sizeof(filename), "file%d.txt", i);
        CHECK_ERROR_SYSCALL(fds[i] = open(filename, O_WRONLY | O_CREAT, 0644));
    }

    1. close_fds();
    2. reset_signal_handlers_and_mask();
    3. sanitize_environment_variables();

    int pipefd[2], pid;
    CHECK_ERROR_SYSCALL(pipe(pipefd));
    CHECK_ERROR_SYSCALL(dup2(pipefd[0], STDIN_FILENO));

    char done;
    4. CHECK_ERROR_STDLIB(pidfile = pidfile_open("/var/run/daemon_name.pid", 0600, NULL));
    CHECK_ERROR_SYSCALL(pid = fork());
```

Daemon Coding Example

```
5. if (pid == 0) {  
    close(pipefd[0]);  
    CHECK_ERROR_SYSCALL(setuid());  
  
    int daemon_pid;  
    6. CHECK_ERROR_SYSCALL(daemon_pid = fork());  
    7. if (daemon_pid == 0) {  
        8. openlog("daemon_name", LOG_PID, LOG_DAEMON);  
  
        9. redirect_to_null_device();  
        10. CHECK_ERROR_SYSCALL(umask(0));  
        11. CHECK_ERROR_SYSCALL(chdir("/"));  
        12. CHECK_ERROR_SYSCALL(pidfile_write(pidfile));  
  
        const char *suid, *sgid;  
        CHECK_ERROR_STDLIB_SYSLOG(suid = secure_getenv("SUDO_UID"));  
        CHECK_ERROR_STDLIB_SYSLOG(sgid = secure_getenv("SUDO_GID"));  
        13. drop_privileges(suid, sgid);  
        14. notify_parent(pipefd[1]);  
        15. CHECK_ERROR_SYSCALL_SYSLOG(pidfile_close(pidfile));  
        16. CHECK_ERROR_SYSCALL_SYSLOG(atexit(handle_exit));  
        17. do_daemon_work();  
    } else return EXIT_SUCCESS;  
} else {  
    close(pipefd[1]);  
    read(pipefd[0], &done, 1);  
    close(pipefd[0]);  
    return EXIT_SUCCESS;  
}
```

Running a daemon

▪ *init*

- The first process launched by Linux that manages all other process, including daemons, that is part of an init system (e.g. SysVinit, openrc, dinit, runit, systemd).
- The default init system used to be SysV init but was eventually replaced with a different init system in most Linux distributions
- Ubuntu uses systemd since Ubuntu 15.04
- systemd mainly has two managers: system and user.
- We will be using the user manager to run our daemons.

Running a daemon

▪ *systemd user manager*

- Manages the services, like daemons, of a user
- To manage a service, a unit file has to be written:

```
[Unit]
Description=Simple Cron Daemon
After=network.target

[Service]
Type=forking
ExecStart=/path/to/simple_cron
RemainAfterExit=yes

[Install]
WantedBy=default.target
```

- This file has to be in `/etc/systemd/user`
 - So, `$ sudo cp simple-cron.service /etc/systemd/user/`

Running a daemon

▪ *systemd user manager*

- After doing this, you have to reload the user manager
 - `$ systemctl --user daemon-reload`
- To start a service, use
 - `$ systemctl --user start simple-cron`
- To stop a service, use
 - `$ systemctl --user stop simple-cron`
- To restart a service, use
 - `$ systemctl --user restart simple-cron`
- To check the status of a service, use
 - `$ systemctl --user status simple-cron`

Final Exam

- **Today is the final lecture! No class next week!**
- **Final Exam: 2025-06-17 (Wed) 18:00~19:00**
 - You have to provide handwritten answers, so please remember to bring something to write with (pencil, pens, etc.).
 - No need to bring your laptop
 - Bring your student ID/identity card (주민등록증, 운전면허증, etc.) and a writing tool (pencil, pen, etc.).
 - Closed-book (no notes), covers all topics
 - Format: multiple-choice, fill-in-the-blanks, etc.

Lab Exercise

■ Create a thread-safe echo daemon

- **Hint:** no skeleton code provided, but echo server implementation can be found in the slides for the 12th week
- No need to implement client (use client code found in week12 or just use ``nc localhost PORT_NUMBER``)
- When a client sends a message, **the server receives the message and sends back the message to the client while logging the message.**
 - » The program name **for syslog (not executable name)** is “STUDENTID_echo” (e.g., 2025000000_echo)
 - » Should log with INFO priority with the following format:
 - » “Echoed N_BYTES bytes to client IP_ADDRESS:PORT_NUMBER”
 - » e.g. Passing “a” may return “Echoed 2 bytes to client 000.000.000.00:50000”
- Create a systemd unit file that runs the executable as a daemon.

Lab Exercise

■ Hint

- Use `getpeername()`, `inet_ntop()`, and `ntohs()` to get the IP address and port.

```
struct sockaddr_in peer_addr;  
socklen_t peer_addr_len = sizeof(peer_addr);  
  
if (getpeername(fd, (struct sockaddr *)&peer_addr, &peer_addr_len) == 0) {  
    char client_ip[INET_ADDRSTRLEN];  
    inet_ntop(AF_INET, &peer_addr.sin_addr, client_ip, sizeof(client_ip));  
    int client_port = ntohs(peer_addr.sin_port);  
}
```

Lab Exercise

■ Example

```
spl on ● SPL in 2025f/w12/w13 via C v13.3.0-gcc
sudo tail -n 6 /var/log/syslog | grep echo
2025-12-02T07:11:35.163527+00:00 hetbed 2025000000_echo: Echoed 4 bytes to client 1xx.xxx.xxx.xx:41530
2025-12-02T07:11:54.018974+00:00 hetbed 2025000000_echo: Echoed 23 bytes to client 1xx.xxx.xxx.xx:41530
2025-12-02T07:11:59.978782+00:00 hetbed 2025000000_echo: Echoed 20 bytes to client 1xx.xxx.xxx.xx:38382
2025-12-02T07:12:07.327404+00:00 hetbed 2025000000_echo: Echoed 23 bytes to client 1xx.xxx.xxx.xx:38382
2025-12-02T07:12:12.034769+00:00 hetbed 2025000000_echo: Echoed 23 bytes to client 1xx.xxx.xxx.xx:41530
2025-12-02T07:12:20.378764+00:00 hetbed 2025000000_echo: Echoed 24 bytes to client 1xx.xxx.xxx.xx:38382

spl on ● SPL in 2025f/w12/w13 via C v13.3.0-gcc
```

```
nc localhost 1398 ~/D/c/s/2/w14

spl on ● SPL in spl/2025f/w14 via C v13.3.0-gcc
nc localhost 1398
Hi!
Hi!
1. First we send this!
1. First we send this!
4. Before sending here
4. Before sending here
```

```
nc localhost 1398 ~/D/c/s/2/w14

spl on ● SPL in spl/2025f/w14 via C v13.3.0-gcc
nc localhost 1398
2. And then this...
2. And then this...
3. And then here again
3. And then here again
5. And now, we're done!
5. And now, we're done!
```

Exercise Submission

- **Submit your source code and Makefile in iCampus**

- Project structure

- » **week14**

- » **Source code (C file)**

- » **systemd unit file (STUDENT_ID-w14.service)**

- » **Makefile**

- » Bundle these with tar comand

- ```
$ tar -caf [student_id]_w14.tar.zst week14
```

- We'll grade your submission with **make**

- » The **make** command should generate **STUDENT\_ID-w14** executable (not w14).

- » If compilation fails, your points for this exercise will be zero.